

Student API with Mongoose ODM

This project is a small Express server that demonstrates how to integrate MongoDB using Mongoose (an Object Data Modeling library). It exposes a simple REST API for managing **students**.

The goal is to help you understand:

- How to connect Node.js/Express to MongoDB with Mongoose
 - How to define a Mongoose schema and model
 - How to build basic CRUD endpoints (Create, Read, Update, Delete)
 - How to use middleware and environment variables
-

1. Project Structure

```
.
├── server.js           # Express app entry point
├── package.json        # Dependencies & scripts
├── config/
│   ├── db.js          # MongoDB connection using Mongoose
│   └── env.js          # Loads environment variables from .env
├── controllers/
│   └── student.controller.js # Request handlers (CRUD logic)
├── middleware/
│   └── logger.js       # Simple request logging middleware
├── models/
│   └── student.model.js # Student Mongoose schema & model
└── routes/
    └── student.routes.js # REST routes for /students
```

2. Prerequisites

- Node.js (v16+ recommended)
 - MongoDB instance (local or cloud, e.g. MongoDB Atlas)
-

3. Setup & Run

1. Install dependencies

```
npm install
```

2. Create a **.env** file in the project root (same level as **server.js**):

```
MONGO_URI=mongodb://localhost:27017/student_db # or your Atlas URI
PORT=5000 # optional, defaults
to 5000
```

3. Start the server

```
npm run dev
```

4. The server will run on:

```
http://localhost:5000
```

All student routes are mounted under `/students`.

4. Student Model (Mongoose)

Defined in `models/student.model.js`.

Main fields:

- `firstName` (String, required, trimmed)
- `lastName` (String, required, trimmed)
- `email` (String, unique, stored in lowercase)
- `cohort` (String, required)
- `enrolledAt` (Date, default: now)
- `gpa` (Number, min: 0, max: 4)
- `interests` (Array of String)

It also shows some extra Mongoose features:

- **Virtuals:** e.g. a `fullName/FullName` virtual combining `firstName` and `lastName`.
- **Instance methods:** e.g. `isHonorStudent()` that checks if `gpa >= 3.5`.
- **Middleware (hooks):** `pre('save')` and `post('save')` that modify/log data around save.

These are useful to understand how Mongoose can encapsulate logic inside the model.

5. API Endpoints

Base URL: `http://localhost:5000/students`

5.1 Create a Student

- **Method:** `POST`
- **URL:** `/students`

- **Body (JSON example):**

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "cohort": "16",
  "gpa": 3.8,
  "interests": ["Node.js", "MongoDB"]
}
```

- **Response:** 201 Created with the created student document.

5.2 Get All Students

- **Method:** GET
- **URL:** /students
- **Response:** Array of student objects.

5.3 Get One Student by ID

- **Method:** GET
- **URL:** /students/:id
- **Response:** Single student object, or 404 if not found.

5.4 Update a Student

- **Method:** PUT
- **URL:** /students/:id
- **Body:** Any fields you want to update (e.g. gpa, interests).
- **Response:** Updated student object, or 404 if not found.

5.5 Delete a Student

- **Method:** DELETE
- **URL:** /students/:id
- **Response:** Message confirming deletion, or 404 if not found.

6. Middleware & Config

- middleware/logger.js logs each request method and URL (e.g. GET /students).
- config/env.js loads environment variables using dotenv from .env.
- config/db.js connects to MongoDB using mongoose.connect(ENV.MONGO_URI, ...).

Understanding this flow is important:

1. server.js loads environment variables.
2. connectDB() is called to connect Mongoose to MongoDB.

3. Express JSON parsing + logger middleware are applied.
 4. Student routes are mounted under `/students`.
-

7. How to Study This Project

To really learn from this code, try these steps:

- Start from `server.js` and follow the flow into `config/db.js`, then into routes and controllers.
- Trace a single request (e.g. `POST /students`) from route → controller → model.
- Experiment by adding new fields to the `studentSchema` and using them in requests.
- Add your own route (for example, `GET /students/cohort/:cohort`) and implement it using Mongoose queries.

This project is meant as a **practice playground** for Express + Mongoose, so feel free to break it, fix it, and extend it.